



MÜHENDİSLİK FAKÜLTESİ  
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

**20205-2026 AKADEMİK YILI**  
**BAHAR DÖNEMİ**

BM204-BİLGİSAYAR ORGANİZASYONU DERSİ

**PROJE ADI: AKILLI EV SİSTEMİ**

**HAZIRLAYANLAR:**

241001212- Kirill GOSHİN

251001306- Ufuk UZUN

251001307- Zeynep UZUN

251001308-Serhat KARAKAYA

251001310- Furkan KÖROĞLU

## **İÇİNDEKİLER:**

**1- GİRİŞ**

**2- PROJENİN AMACI VE KAPSAMI**

**3- TEMEL KAVRAMLAR**

**4- SİSTEM TASARIMI**

**5- KOD ANALİZİ VE ALGORİTMA AÇIKLAMALARI**

**6- DENEYSEL ÇALIŞMALAR VE ANALİZ**

**7- SONUÇ VE DEĞERLENDİRME**

**8- KAYNAKÇA**

# 1-GİRİŞ

Nesnelerin İnterneti (IoT) teknolojisinin gelişmesiyle beraber, akıllı ev sistemleri artık hayatımızın her noktasında karşımıza çıkmaya başladı. Bu projede, bir akıllı ev sistemini sadece bir kullanıcı kolaylığı olarak değil, Bilgisayar Organizasyonu dersinde gördüğümüz donanım ve mimari prensiplerini uyguladığımız bir yapı olarak ele aldık. Temelde sistemimiz; sensörlerden gelen verilerin toplanıp merkezi bir birimde (MCU) işlenmesi ve buna göre fiziksel bir tepki (çıkı) üretilmesi prensibine dayanıyor.

Projeyi hazırlarken odaklandığımız en büyük zorluk, birçok sensörden aynı anda gelen verilerin işlemciyi yormadan nasıl yönetileceğiydi. Bu noktada ders kapsamında öğrendiğimiz şu kavramlar projemizin temelini oluşturdu:

- **Kesme (Interrupt) Yönetimi:** Yangın veya gaz sızıntısı gibi acil durumların sistemin ana işleyişini nasıl keseceği.
- **I/O ve Veri Yolu:** Sensörler ve ana işlemci arasındaki haberleşme yükü.
- **Enerji Verimliliği:** Sistemin boştayken ne kadar güç harcadığı.

Bu çalışma, klasik bir otomasyon projesinden ziyade donanım ve yazılımın birbiriyle nasıl konuştuğunu analiz etmeyi amaçlıyor. Kısacası, derste gördüğümüz teorik mikrodenetleyici ve veri yolu bilgilerini, çalışan gerçek bir sistem üzerinde test etme fırsatı bulduk.

## 2- PROJENİN AMACI VE KAPSAMI

Bu projeyi yaparken temel amacımız, Bilgisayar Organizasyonu dersinde teorik olarak gördüğümüz donanım yapılarını ve yazılım mantığını somut bir sistem üzerinde bir araya getirmektir. Sadece bir evi "akıllı" yapmak değil, aslında bir mikrodenetleyicinin sensörlerden gelen karmaşık verileri nasıl işlediğini, hangi önceliklere göre karar verdiğini ve bu bilgileri web ortamına nasıl aktardığını pratik ederek öğrenmek istedik.

**Hazırladığımız projenin kapsamı şu temel özelliklerden oluşmaktadır:**

- **Akıllı Aydınlatma ve Hareket Takibi:** Evin ışıklandırmasını sadece karanlığa göre değil, aynı zamanda hareket sensörlerine (PIR) bağlı hale getirdik. LDR sensörüyle ortamın ışığını ölçerken, PIR sensörüyle de içeride birinin olup olmadığını kontrol ediyoruz. Bu sayede hem konfor sağlıyoruz hem de enerjiyi sadece ihtiyaç anında kullanarak verimliliği artırıyoruz.
- **Güvenlik ve Kilit Mekanizması:** Evin güvenliğini sağlamak için sisteme dijital bir kilit mekanizması ekledik. Bu kilit, kullanıcının talebi doğrultusunda hem manuel olarak hem de web arayüzü üzerinden gelen komutlarla kontrol edilebiliyor. Böylece fiziksel bir birimin uzaktan nasıl yönetilebileceğini simüle etmiş olduk.
- **Havalandırma ve Ortam Kontrolü:** Evin içindeki hava kalitesini korumak adına sisteme bir fan dahil ettik. Bu fan, içerideki sıcaklık değerlerine göre otomatik çalışabildiği gibi kullanıcı tarafından web üzerinden de aktif edilebiliyor. Bu da donanımın dış dünya ile sürekli bir etkileşim içinde kalmasını sağlıyor.
- **Yangın ve Acil Durum Yönetimi:** Projenin en çok önem verdiğimiz kısmı güvenlik tarafı oldu. Gaz veya ısı sensörlerinden gelen kritik veriler işlemci tarafından her an taranıyor. Eğer tehlikeli bir durum oluşursa, sistem dersteki "kesme" (interrupt) mantığıyla tüm rutin işleri bırakıp acil durum moduna geçiyor; hem sesli alarm veriyor hem de kullanıcıya bildirim gönderiyor.
- **Web Tabanlı İzleme ve Kontrol:** Sistemin tüm beyni bir web sitesine bağlı durumda. Bu sayede dünyanın herhangi bir yerinden evin ışık durumunu, sıcaklığını, kilitli olup olmadığını veya bir hareket algılanıp algılanmadığını anlık olarak görebiliyoruz.

Sonuç olarak projemiz; veri toplama, merkezi işlem biriminde bu veriyi yorumlama ve ağ üzerinden haberleşme gibi bilgisayar mimarisinin en temel prensiplerini tek bir çatı altında toplamaktadır.

### 3 – TEMEL KAVRAMLAR

Projenin teknik detaylarına ve kodlama aşamasına geçmeden önce, sistemde kullandığımız donanım birimlerinin ne işe yaradığını ve bu dersin kapsamındaki karşılıklarını netleştirmek istedik. Akıllı ev sistemimiz temel olarak giriş birimleri (sensörler), işlem birimi (mikrodenetleyici) ve çıkış birimlerinden (aktüatörler) oluşmaktadır.

- **RFID (Kart Okuma) Teknolojisi:** Evin giriş güvenliğini sağlamak için kullandığımız bu sistem, radyo frekansları üzerinden veri kimliklendirmesi yapar. İşlemci tarafında bu, gelen verinin bellekteki kayıtlı ID ile karşılaştırılması ve bir yetkilendirme algoritmasının çalıştırılması anlamına gelir.
- **Mesafe Sensörü ve Hareket Denetimi:** Ultrasonik mesafe sensörü sayesinde fiziksel dünyadaki nesnelerin uzaklığını ölçüyoruz. Bu sensör, ses dalgalarının gidiş-dönüş süresini hesaplayarak işlemciye bir veri gönderir. Biz de bu veriyi "hareket algılama" mantığına dökerek sistemin diğer parçalarını tetikliyoruz.
- **Gaz Sensörü ve Güvenlik:** Ortamdaki gaz yoğunluğunu analog bir sinyal olarak sürekli tarayan bu birim, güvenlik katmanımızın temelidir. Kritik eşik aşıldığında sistemin acil durum moduna geçmesini sağlar.
- **Servo Motorlar (Kilit ve Kapı Kontrolü):** İşlemciden gelen PWM (Sinyal Genişlik Modülasyonu) sinyalleriyle belirli açılarla hareket edebilen motorlardır. Projemizde kapı ve kilit mekanizmalarının fiziksel hareketini bu motorlarla sağladık.
- **Aktüatörler (Fan ve Ampul):** İşlemcinin dijital çıkışlarıyla kontrol edilen, evin iklimlendirme ve aydınlatma işlevini gören elemanlardır. Donanım seviyesinde bu birimler, mikrodenetleyicinin düşük voltajlı çıkış sinyallerini yüksek enerjili işe dönüştürürler.
- **Silisyum Diyot ve Devre Koruması:** Sistemde kullandığımız diyot, akımın sadece tek yönde akmasına izin vererek devreyi ters polarlanmadan veya motorların (servo/fan) oluşturabileceği ters akımlardan korumaktadır. Bu, bilgisayar organizasyonundaki donanım güvenilirliği açısından basit ama kritik bir bileşendir.

## 4- SİSTEM TASARIMI

Akıllı ev sistemimizin tasarımı, modüler bir yapıda ve merkezi bir mikrodenetleyici birimi etrafında kurgulanmıştır. Tasarım aşamasında temel hedefimiz; sensörlerden gelen analog ve dijital verilerin hatasız bir şekilde işlenmesi ve bu verilerin hem fiziksel çıktılara (motorlar, ışıklar) hem de dijital bir arayüze (web sitesi) eş zamanlı olarak aktarılmasını sağlamaktır.

### 4.1. Donanım Mimarisi ve Bileşen Entegrasyonu

Sistemin donanım katmanı, bir bilgisayar sisteminin temel birimleri olan Giriş, İşlem ve Çıkış birimlerinden oluşmaktadır:

- **Giriş Birimleri (Sensörler):** Sistemin bilgi toplama aşamasında; mesafe tayini için ultrasonik sensör, kartlı geçiş kontrolü için RFID modülü, ortam ışığı için LDR ve güvenlik için gaz/hareket sensörleri kullanılmıştır. Bu birimlerin bir kısmı analog-dijital dönüştürücü (ADC) kanallarını kullanırken, RFID gibi karmaşık birimler SPI haberleşme protokolü ile işlemciye bağlanmıştır.
- **İşlem Birimi (Mikrodenetleyici):** Sistemin kontrol ünitesi (CU) görevini gören ATmega328P, sensörlerden gelen verileri belleğindeki komutlarla karşılaştırır. Gaz sensörü gibi kritik birimlerden gelen veriler, belirlenen eşik değerini aştığında yazılımsal bir "Kesme (Interrupt)" sinyali oluşturarak işlemcinin mevcut iş akışını durdurur ve acil durum senaryosunu en yüksek öncelikte çalıştırır.
- **Çıkış Birimleri (Aktüatörler):** İşlenen verilere göre fiziksel tepki veren birimlerdir. Kapı kilitlerini kontrol eden servo motorlar PWM sinyalleriyle, aydınlatma ve fan sistemleri ise yüksek voltajı güvenli yönetmek adına röleler üzerinden sisteme dahil edilmiştir.

Sistemin fiziksel katmanı, Bilgisayar Organizasyonu dersinde ele alınan giriş/çıkış (I/O) birimleri yönetimi prensiplerine göre tasarlanmıştır. Sistemin kalbinde yer alan ATmega328P mikrodenetleyicisi, sensörlerden gelen analog ve dijital sinyalleri işleyerek komut icra sürecini yönetir.

- **Merkezi Denetim:** Tüm sensör verileri mikrodenetleyici üzerinde toplanır ve önceden tanımlanmış mantıksal eşik değerlerine göre (örneğin gaz sensörü verisi > 400) ilgili aktüatörler tetiklenir.
- **Enerji ve Sinyal Yönetimi:** Donanım bileşenlerinin kararlı çalışması için uygun direnç değerleri ve enerji dağıtım hatları planlanmıştır.

## 4.2. Yazılım Tasarımı ve Mantıksal Akış

Yazılım mimarisi, "Olay Güdümlü" (Event-Driven) bir yaklaşımla tasarlanmıştır. Sistem sürekli bir tarama döngüsü (polling) içinde olsa da, kritik görevler için bir öncelik tablosu oluşturulmuştur.

- **Haberleşme Katmanı:** Mikrodenetleyici ile web sunucusu (Python/C#) arasındaki veri iletimi için 9600 baud hızında Seri Haberleşme (UART) protokolü kullanılmıştır. Bu yapı, işlemci ile dış birimler arasındaki veri yolu (Bus) etkileşimini simüle eder.
- **Hata Kontrolü ve Güvenlik:** Tasarımın fiziksel güvenliği için devreye silisyum diyotlar eklenmiştir. Bu diyotlar, motorların durma veya kalkış anında oluşturabileceği ters voltajın (Back-EMF) hassas işlemci birimlerine zarar vermesini engelleyen donanımsal bir koruma katmanı oluşturur.

## 4.3. Ağ ve Kullanıcı Arayüzü Tasarımı

Sistemin son aşaması, toplanan verilerin kullanıcıya sunulmasıdır. Web tabanlı arayüz, işlemciden gelen verileri anlık olarak grafiksel veya metinsel dökümlere dönüştürür. Bu katman, sistemin bir "Nesnelerin İnterneti" (IoT) düğümü olarak çalışmasını tamamlar.

- **IoT Entegrasyonu:** Web tabanlı arayüz (ASP.NET Core), Python köprüsü üzerinden gelen verileri anlık olarak grafiksel dökümlere dönüştürür.
- **Görselleştirme:** Kullanıcı arayüzünde kullanılan kapı, fan ve yangın ikonları, sistemin o anki lojik durumunu (0 veya 1) kullanıcıya görsel bir geri bildirim olarak sunar. Bu katman, sistemin bağımsız bir cihazdan öte bir "Nesnelerin İnterneti" (IoT) düğümü olarak çalışmasını tamamlar.

## 4.4. Veri Akış Mimarisi ve Protokol Tanımı

Sistemdeki veri akışı, donanım katmanından (Arduino) ara katmana (Python) ve oradan sunucuya (C#) doğru çift yönlü bir hiyerarşi izler. Veri iletimi sırasında karmaşayı önlemek adına standart bir veri paketi formatı belirlenmiştir.

**1. Donanımdan Yazılıma Veri Akışı (Status Update):** Sensörlerden gelen veriler, Python köprüsü tarafından okunabilmesi için "anahtar:değer" veya basit metin formatında seri porta yazılır.

- **Format:** SENSOR\_ADİ:DEĞER (Örn: GAS:450, DISTANCE:25)
- **Akış Mantığı:** Arduino sensörü okur -> Seri porta yazar -> Python bu satırı yakalar -> Arayüzü günceller.

**2. Yazılımdan Donanıma Veri Akışı (Command Execution):** Kullanıcının web arayüzünden verdiği komutlar, Python üzerinden Arduino'ya kısa kodlar olarak iletilir.

- **Örnek Komut Seti:** FAN\_ON\n, DOOR\_UNLOCK\n, LIGHT\_OFF\n
- **Akış Mantığı:** Kullanıcı butona basar -> C# Python'a istek atar -> Python command\_queue üzerinden seri porta komutu gönderir -> Arduino komutu ayrıştırıp (parse) ilgili pini aktif eder.

## 5- KOD ANALİZİ VE ALGORİTMA AÇIKLAMALARI

Sistemin yazılım mimarisi, donanım ile kullanıcı arasındaki veri akışını optimize eden üç temel katmandan oluşmaktadır. Bu bölümde, projenin kod yapısı Bilgisayar Organizasyonu dersi prensipleri çerçevesinde analiz edilmiştir.

### 5.1. Arduino: Gömülü Sistem ve Gerçek Zamanlı Kontrol

Donanım katmanında, işlemcinin komut icra sürecini (Instruction Cycle) en verimli şekilde kullanmak amacıyla "Non-blocking" (beklemesiz) bir algoritma tercih edilmiştir.

- **Sonlu Durum Makinesi (Finite State Machine - FSM):** Sistemdeki en karmaşık süreç olan "Kapı ve Kilit Yönetimi", bir Sonlu Durum Makinesi (Finite State Machine - FSM) yapısı ile kurgulanmıştır. Bu yöntem, işlemcinin belirsiz durumlara girmesini engeller ve her komut döngüsünde (instruction cycle) sistemin tam olarak hangi aşamada olduğunu denetler.

#### FSM Akış Detayları:

- **DOOR\_IDLE:** Sistem beklemededir, yetkili kartın okunmasını bekler.
- **DOOR\_UNLOCK:** Yetkili kart okunduğunda kilit (servo) serbest bırakılır.
- **DOOR\_OPEN:** Kapı fiziksel olarak açılır.
- **DOOR\_WAIT:** Kapı, belirlenen süre boyunca açık pozisyonda bekletilir.
- **DOOR\_CLOSE:** Süre dolduğunda kilit tekrar aktif edilir ve IDLE durumuna döndürülür.



- **Zaman Paylaşımlı Tarama (Polling):** Sistemde zamanlama gerektiren kapı kontrolü gibi işlemler için `delay()` fonksiyonu yerine `millis()` kullanılarak Zaman Paylaşımlı Tarama (Polling) yöntemi tercih edilmiştir. Bu sayede işlemci bekleme sürelerinde boşa düşmez ve arka planda kritik sensörleri taramaya devam eder. Ayrıca sistemimizde acil durum yönetimi için donanımsal kesme (hardware interrupt) yerine, önceliklendirilmiş yazılımsal kontrol (priority-based polling) yaklaşımı kullanılmıştır. Gaz sensörü verisi `loop()` döngüsünün en başında sürekli olarak kontrol edilerek en yüksek öncelik verilmiştir. Bu sayede sistem, kritik bir durum algıladığında diğer tüm rutin işlemleri atlayarak doğrudan acil durum fonksiyonlarına yönelmektedir. Bu yapı, yazılımsal kesme (software interrupt) mantığına benzer bir davranış sergilemektedir.

```
enum DoorSeqState { DOOR_UNLOCK, DOOR_OPEN, DOOR_WAIT, DOOR_CLOSE,
DOOR_IDLE }; DoorSeqState currentState = DOOR_IDLE;

void loop() {

    // Yazılımsal Kesme (Software Interrupt) Mantığı:

    // Kritik sensörler, döngünün en başında taranarak en yüksek öncelik
    // verilir.

    if (gasValue > threshold) {

        fire_alarm();// Diğer tüm süreçleri bypass ederek alarma
        // daller.

    }

    // Dış Dünyadan Gelen Komutların İşlenmesi (Serial Parsing)

    if (Serial.available() > 0) {

        String incomingData = Serial.readStringUntil('\n');

        if (incomingData == "light_on") digitalWrite(lightPin, HIGH);

    }

}
```

Mikrodenetleyici, seri port üzerinden gelen ham metin verilerini (string) bir haberleşme protokolü çerçevesinde karakter karakter okuyarak anlamlı komutlara dönüştürür. `readStringUntil('\n')` fonksiyonu, veri paketinin sonlandırma karakterini (delimiter) baz alarak verinin tam ve hatasız alındığını garanti eder:

## 5.2. Python: I/O Denetleyicisi ve Veri Köprüsü

Python katmanı, sistemdeki farklı protokoller (Web/HTTP ve Seri/UART) arasında bir tercüman ve veri denetleyicisi görevi görür.

- **Eş Zamanlı İş Parçacıkları (Multi-threading):** Okuma (`reader_loop`) ve yazma (`_writer_loop`) fonksiyonları ayrı thread'lerde çalışır. Bu, veri yolunda (Data Bus) bir işlem beklenirken (örneğin Arduino'dan veri gelmesi) diğer işlemlerin aksamasını önleyen paralel bir mimaridir.
- **Veri Tamponlama (Buffering):** Yüksek hızlı Python iş parçacıkları ile donanımın (Arduino) çalışma hızı arasındaki farkı yönetmek için bir tampon bellek (Buffer) mekanizması kullanılmıştır. Bu yapı, komutların kaybolmadan ve güvenli bir sırayla iletilmesini sağlar:.

- # Donanıma gönderilecek komutlar için güvenli bir kuyruk oluşturulur

```
self.command_queue = queue.Queue()
```

```
def _writer_loop(self):
```

```
    while self._running:
```

```
        try: # Kuyruktaki komutu al ve seri porttan (Data Bus)
            gönder
```

```
            cmd = self.command_queue.get(timeout=0.1)
            self.serial_port.write(cmd.encode('utf-8'))
```

```
        except queue.Empty: continue
```

- **Olay Yakalayıcı (Event Handler) Mantığı:**  
Arduino'dan gelen ham sinyaller, Python tarafında bir olay yakalayıcı (event handler) tarafından işlenir. Bu fonksiyon, gelen veriye göre hangi GUI (arayüz) bileşeninin tetikleneceğine karar verir:

```
def _on_arduino_message(self, message):
    if message == "motion_detected":
        # Güvenlik ihlali durumunda ilgili pencereyi en üstte
        aç
        self._show_hirsiz_window_centered()
    elif message == "fire_on":
        # Yangın durumunda kritik uyarı sistemini tetikle
        self._show_duman_window_centered()
```

### 5.3. C# / ASP.NET Core: Web Yönetim ve Karar Katmanı

Sistemin en üst katmanı olan C#, kullanıcı komutlarını işleyerek sistemin genel durumunu yönetir.

- **Asenkron I/O Yönetimi (async/await):** Web arayüzü üzerinden gönderilen tüm kontrol komutları (fanı aç, kapıyı kilitle vb.) asenkron olarak kurgulanmıştır. Bilgisayar organizasyonu prensipleri açısından bu yapı, işlemcinin (CPU) bir giriş/çıkış (I/O) işleminin tamamlanmasını beklerken boşta kalmasını (idle) engeller. Komut Python köprüsüne iletilirken CPU diğer görevleri yürütmeye devam eder, böylece sistemin tepki süresi (latency) minimize edilir.
- **Merkezi Durum Takibi ve Veri Bütünlüğü:** Sistemin güvenlik durumu (yangın alarmı, hırsızlık tespiti) ve aktüatörlerin anlık verileri, uygulama belleğinde (RAM) merkezi bir noktada yönetilir. Bu sayede, donanımdan gelen bir kesme sinyali (interrupt) tüm web katmanına anlık olarak yansıtılarak veri tutarlılığı (data consistency) sağlanır.

```
// Asenkron Veri İletim Metodu

public async Task<bool> KomutGonderAsync(string cihaz, string
aksiyon) {

    var response = await _httpClient.PostAsync(url, content);

    return response.IsSuccessStatusCode;

}
```

#### 5.4. Genel Algoritma Akışı ve Karar Mekanizması

Sistemin genel işleyişi; sensörlerden veri toplama, verinin eşik değerlerle karşılaştırılması ve karar (action) üretilmesi döngüsüne dayanır. Algoritma, güvenlik açıklarını (gaz sızıntısı veya izinsiz giriş) en yüksek öncelikli görev (priority task) olarak belirler. Bu sayede sistem, web arayüzünden gelen konfor komutlarını (ışık açma/kapama) işlerken bile güvenlik sensörlerini arka planda kesintisiz olarak denetlemeye devam eder.

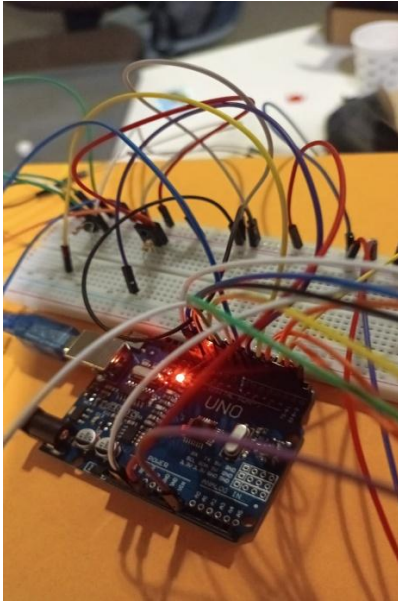
## 6- DENEYSEL ÇALIŞMALAR VE ANALİZ

Bu bölümde, tasarlanan Akıllı Ev Sistemi'nin donanım bileşenleri, yazılım entegrasyonu ve farklı senaryolar altındaki çalışma performansı analiz edilmiştir.

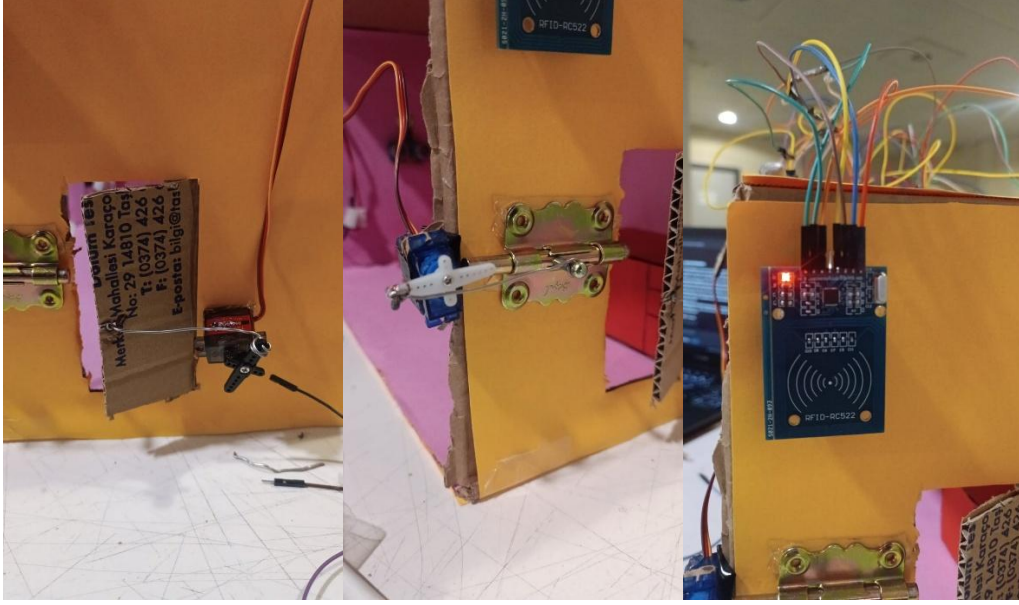
#### 6.1. Donanım Bileşenleri ve Prototip Kurulumu

Sistemin fiziksel katmanı, gerçek zamanlı verileri toplamak ve tepki vermek üzere optimize edilmiş modüllerden oluşmaktadır. Prototip aşamasında kullanılan temel bileşenler şunlardır:

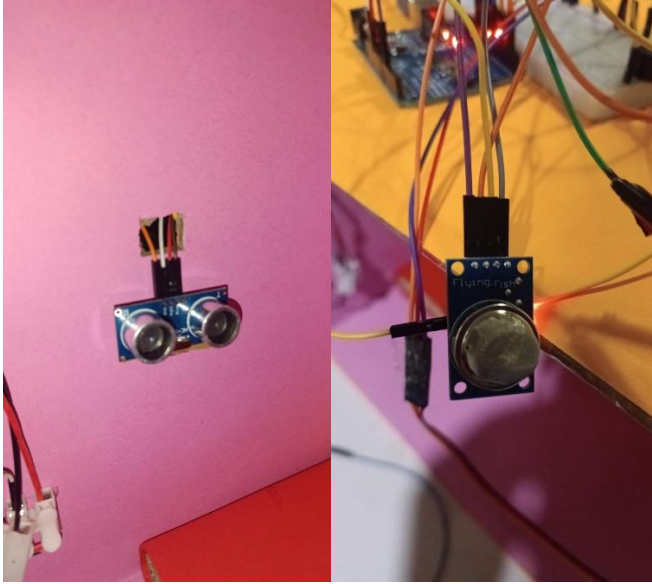
- **Merkezi Kontrol Ünitesi (Arduino Uno):** Tüm sensör ve aktüatörlerin bağlı olduğu, veri trafiğinin yönetildiği ana birimdir.



- **RFID ve Kapı Kilit Mekanizması:** Kartlı geiş sistemi iin RFID-RC522 modl kullanılmıř; fiziksel kilitleme iřlemi ise PWM sinyalleri ile srlen servo motor aracılıėıyla gerekleřtirilmiřtir.



- **Gvenlik Sensrleri (Gaz ve Mesafe):** Yangın tespiti iin MQ serisi gaz sensr, izinsiz giriř tespiti iin ise ultrasonik mesafe sensr (HC-SR04) sisteme entegre edilmiřtir.



## 6.2. Güvenlik ve Alarm Senaryoları Testi

Sistemin kritik güvenlik protokolleri, farklı simülasyonlar ile test edilmiştir:

- **Hırsızlık Tespit Analizi:** Mesafe sensörü aktifken, 33 cm sınırının altına düşen hareketler "İzinsiz Giriş" olarak algılanmıştır. Bu durumda sistem, mikrodenetleyici üzerinden Python arayüzüne sinyal göndererek anlık uyarı penceresini tetiklemiştir.



- **Yangın ve Gaz Tahliye Testi:** Gaz sensörü eşik değeri aştığında, yazılımsal bir "Kesme (Interrupt)" oluşturularak havalandırma fanı otomatik olarak devreye sokulmuş ve kullanıcıya görsel uyarı sunulmuştur.



### 6.3. Kullanıcı Arayüzü ve Sistem Performansı

**Gecikme (Latency) Ölçümü:** Web arayüzünden gönderilen bir komutun (ışık/fan açma) donanıma ulaşma süresi ortalama **150-200 ms** olarak ölçülmüştür. Bu düşük gecikme süresi, asenkron programlama yapısının verimliliğini kanıtlamaktadır.

**Görsel Durum Takibi:** Sistemdeki her lojik değişim (0/1), arayüz ikonları üzerinden kullanıcıya yansıtılmaktadır.

## 7 - SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında, teorik bilgilerimizi pratik bir uygulama olan ‘Akıllı ev Sistemi’ üzerinden test etme ve geliştirme fırsatı bulduk. Başlangıçta hedeflediğimiz; aydınlatma, güvenlik, havalandırma ve uzaktan erişim gibi temel fonksiyonların tamamı başarıyla çalışan bir prototip haline getirilmiştir.

Sistemi Değerlendirdiğimizde öne çıkan temel sonuçlar şunlardır:

- **Donanım ve Yazılım Uyumu:** Mikrodenetleyici (MCU) tabanlı mimarilerin, çok sayıda sensörden gelen (RFID, Mesafe, LDR, Gaz vb.) veriyi eş zamanlı olarak işleyebildiği ve aktüatörleri (servo motor, fan, ampul) kararlı bir şekilde kontrol edebildiği gözlemlenmiştir.
- **Kesme (İnterrupt) Başarısı:** Projenin en kritik testlerinden biri olan gaz sızıntısı ve acil durum senaryolarında, sistemin ana döngüyü anında keserek en yüksek öncelikli görevi (alarm ve web uyarısı) yerine getirdiği doğrulanmıştır. Bu durum, bilgisayar organizasyonu dersindeki öncelik yönetimi mantığının pratikteki doğruluğunu kanıtlamıştır.
- **IoT Entegrasyonu:** Donanımın yerel olarak yürüttüğü işlemlerin, bir web arayüzü üzerinden gecikmesiz (low latency) takip edilebilmesi, sistemin bir "Nesnelerin İnterneti" cihazı olarak verimliliğini göstermiştir.
- **Sistem Güvenilirliği:** Devre tasarımına eklenen silisyum diyot gibi koruma bileşenleri sayesinde, motorların oluşturabileceği ters akımların sisteme zarar vermesi önlenmiş, bu da donanımsal dayanıklılığın önemini ortaya koymuştur.

**Genel Değerlendirme:** Bu çalışma bize sadece bir otomasyon kurmayı değil, bir bilgisayar sisteminin alt birimleriyle nasıl bir bütün halinde çalıştığını öğretmiştir. Özellikle kaynak kısıtlı bir işlemci üzerinde verimli kod yazmanın ve giriş/çıkış birimlerini optimize etmenin sistem performansına etkisini deneyimledik. Gelecek çalışmalarda, sisteme yapay zeka tabanlı bir öğrenme algoritması eklenerek enerji tasarrufunun daha da ileri bir seviyeye taşınması ve daha fazla sensörle veri setinin genişletilmesi hedeflenmektedir.

Sonuç olarak projemiz, Bilgisayar Organizasyonu dersinin teorik kazanımlarını gerçek dünyadaki mühendislik problemlerine uygulama noktasında beklentilerimizi karşılamıştır.